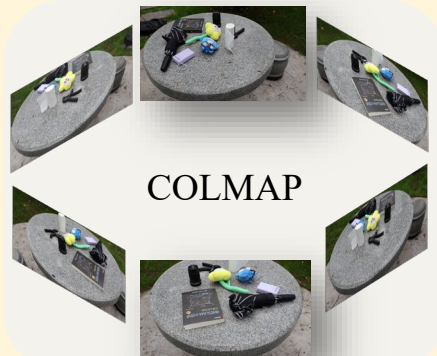


代表项目：LangSplat跨模态3D场景语义理解

数据采集和初始化

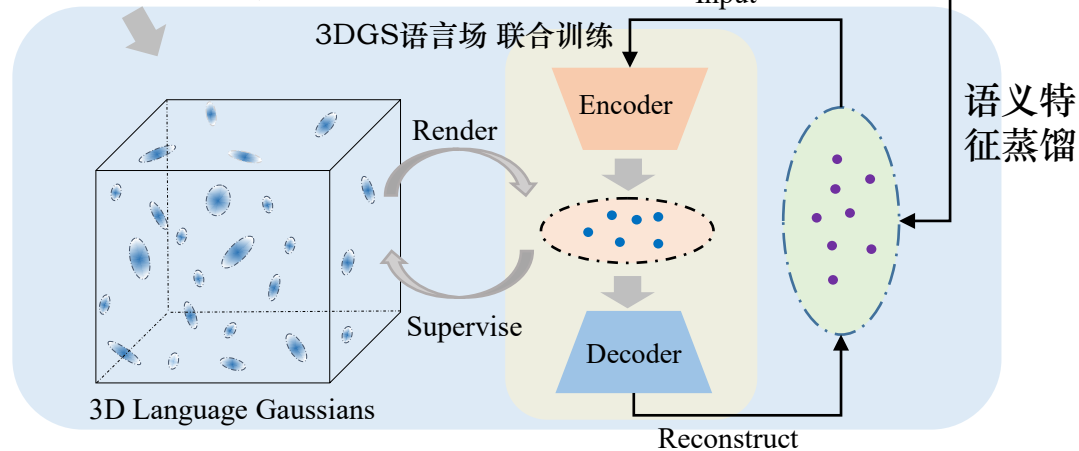


COLMAP

初始稀疏点云



Vanilla 3DGS 预训练



SAM精细掩码分割

SAM



语义提取

CLIP



➤ Original GT



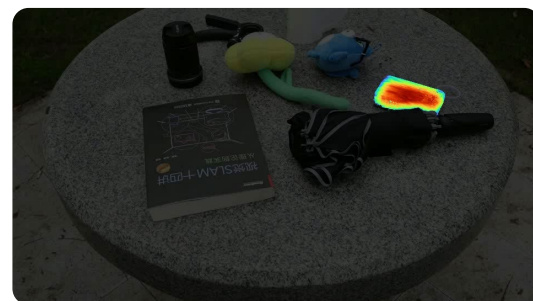
➤ Vanilla 3DGS



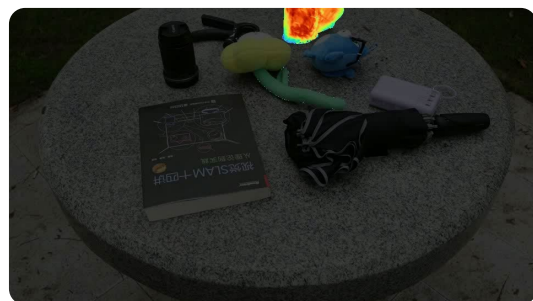
➤ a book



➤ a blue plush toy



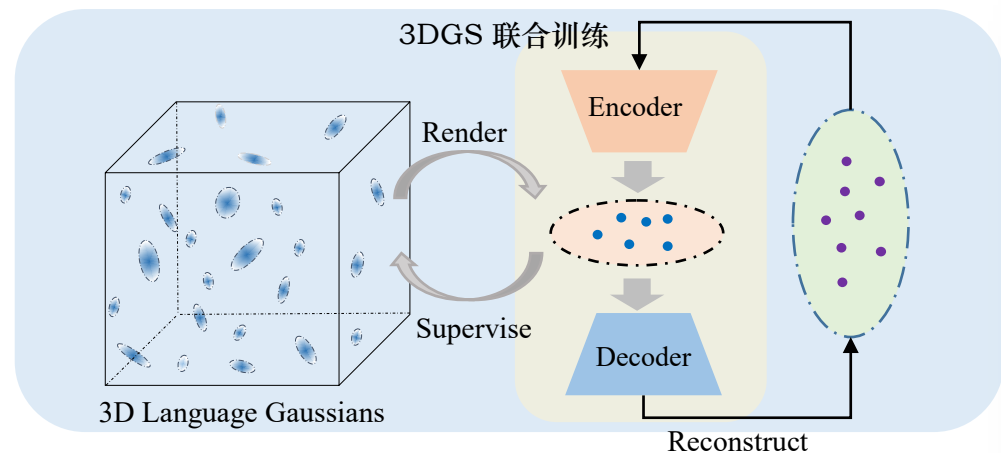
➤ a power bank



➤ a toilet paper



核心挑战一：跨语言接口触发的 CUDA 内存访问异常



```
(LangSplat) root@autodl-container-ef19489c10-0cf4e1c9:~/autodl-tmp/3DGS_Project/LangSplat/LangSplat$ python train.py -s ./data/my_desk -m ./output/my_desk_vanilla -r 4
Namespace(sh_degree=3, source_path='./data/my_desk', model_path='./output/my_desk_vanilla', language_features_name='language_features_dim3', images='images', resolution=4, white_b
background=False, feature_level=-1, data_device='cuda', eval=False, iterations=30000, position_lr_init=0.00016, position_lr_final=1.6e-06, position_lr_delay_mult=0.01, position_lr
max_steps=30000, feature_lr=0.0025, opacity_lr=0.05, language_feature_lr=0.0025, include_feature=False, scaling_lr=0.005, rotation_lr=0.001, percent_dense=0.01, lambda_dssim=0.2,
densification_interval=100, opacity_reset_interval=3000, densify_from_iter=500, density_until_iter=15000, densify_grad_threshold=0.0002, convert_Sha_python=False, compute_cov3D_p
thon=False, debug=False, ip='127.0.0.1', port=55555, debug_from=-1, detect_anomaly=False, test_iterations=[7000, 30000], save_iterations=[7000, 30000, 30000], quiet=False, checkpo
int_iterations=[7000, 30000], start_checkpoint=None)
Optimizing ./output/my_desk_vanilla -1
Output folder: ./output/my_desk_vanilla -1 [26/03 18:28:03]
Reading camera 248/248 [26/03 18:28:05]
Loading Training Cameras [26/03 18:28:06]
Loading Test Cameras [26/03 18:28:04]
Number of points at initialisation : 260195 [26/03 18:28:04]
Training progress: 0% [0/30000 [00:00<7, 21t/s]]
Traceback (most recent call last):
  File "/root/autodl-tmp/3DGS_Project/LangSplat/LangSplat/train.py", line 231, in <module>
    training(lp.extract(args), op.extract(args), pp.extract(args), args.test_iterations, args.save_iterations, args.checkpoint_iterations, args.start_checkpoint, args.debug_from)
  File "/root/autodl-tmp/3DGS_Project/LangSplat/LangSplat/train.py", line 104, in training
    loss.backward()
  File "/root/miniconda3/envs/LangSplat/lib/python3.10/site-packages/torch/_tensor.py", line 492, in backward
    torch.autograd.backward(
  File "/root/miniconda3/envs/LangSplat/lib/python3.10/site-packages/torch/autograd/_init_.py", line 251, in backward
    Variable_execution_engine.run_backward(*args)
RuntimeError: CUDA error: an illegal memory access was encountered
CUDA kernel errors might be asynchronously reported at some other API call, so the stacktrace below might be incorrect.
For debugging consider passing CUDA_LAUNCH_BLOCKING=1.
Compile with 'TORCH_USE_CUDA_DSA' to enable device-side assertions.

Training progress: 0% [0/30000 [00:00<7, 21t/s]]
```

RuntimeError: CUDA error: an illegal memory access

include_feature=False

- 异常现象：在进行 **Vanilla** 几何预训练时（关闭语言特征 `include_feature=False`），程序在反向传播阶段触发 **Illegal memory access** 导致进程崩溃。
- 排查过程：开启 `CUDA_LAUNCH_BLOCKING=1` 强制执行同步，将报错精准定位至底层高斯光栅化（**Rasterization**）算子。检查该算子的前置输入，发现 Python 层传入的语言特征占位张量维度仅为 **(1,)**。
- 根因分析：底层 C++ 反向传播核函数对该特征关闭分支缺乏有效的条件拦截。CUDA 启动了与点云数量一致的并发线程进行寻址，导致访问 **(1,)** 张量时发生严重的显存越界。



跨语言接口张量安全对齐

```
# gaussian_renderer/__init__.py (原始代码)
if opt.include_feature:
    # 正常训练分支: 传入完整的语言特征张量
    language_feature_precomp = pc.get_language_feature
else:
    # 致命缺陷分支: 当关闭语言特征时 (include_feature=False)
    # Python层仅分配了 size=(1,) 的微小内存作为签名占位符。
    # 导致底层 C++ 反向传播时, 26万个并发线程读取越界,
    # 触发 Segfault 硬件崩溃。
    language_feature_precomp = torch.zeros((1,),
                                           dtype=opacity.dtype, device=opacity.device)
```

原始代码框架: 零假向量占位导致底层并发寻址越界

```
# gaussian_renderer/__init__.py (重构修复)
if opt.include_feature:
    language_feature_precomp = pc.get_language_feature
else:
    # 修复: 放弃深入 CUDA 源码打补丁, 直接在顶层执行张量广播(Broadcast)
    # 动态获取当前高斯点云数量 N = pc.get_xyz.shape[0]
    num_points = pc.get_xyz.shape[0]

    # 强制将占位张量的维度对齐至 (N, 3)
    # 即使底层线程在 backward() 中误读,
    # 也能安全访问合法的零值内存, 彻底阻断崩溃。
    language_feature_precomp = torch.zeros((num_points, 3),
                                           dtype=opacity.dtype,
                                           device=opacity.device)
```

基于 Broadcast 机制的顶层张量对齐

核心目标是消除高层传入的占位张量与底层 CUDA 并发线程数之间的 Shape 错位。

➤ 优化思路: 选择在 Python 接口层进行数据规范化, 保证传入底层的特征张量维度与实际点云规模严格匹配。

➤ 具体实现:

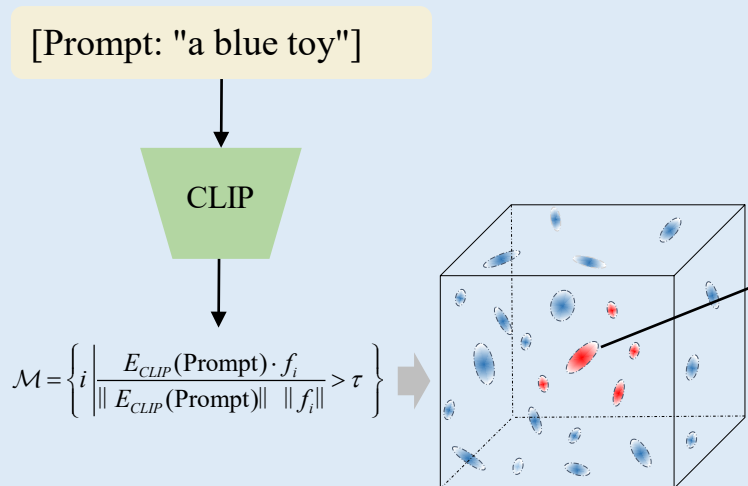
- 动态获取当前迭代步骤中的高斯点云总数 N ($\text{num_points} = \text{pc.get_xyz.shape}[0]$)。
- 利用 $\text{torch.zeros}(N, 3)$ 替换原有的 $(1,)$ 占位张量, 进行维度广播 (Broadcast)。

➤ 最终效果:

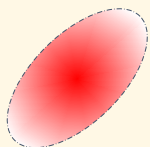
使得底层的并发线程在反向传播时能够安全读取合法的零值内存。在维持框架整体结构的条件下, 消除了段错误隐患, 成功跑通端到端预训练管线。

基于已有语义场的物理级场景干预

语义检索层



参数设置层



μ 位置
 Σ 协方差/形状
 c 颜色
 $language\ feature$ 语义
 α 不透明度

$$\sigma \quad \sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\alpha_{final}$$

物理渲染层



$$C = \sum_{i=1}^N c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j)$$



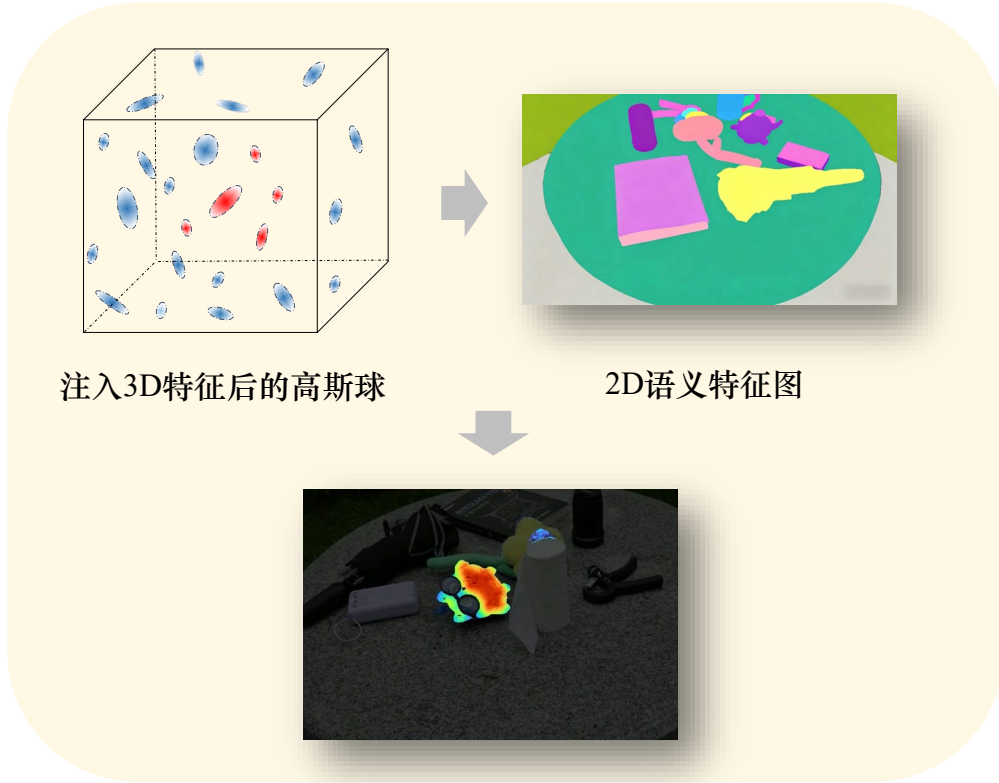
- 在 LangSplat 的设定中，每个 3D 高斯不仅包含几何参数（中心位置、协方差），还绑定了一个额外的高维语义向量（Language Feature）。
- 通过计算 Prompt 向量与所有高斯球语义向量的余弦相似度,设定阈值，精准圈定目标物体的 3D 索引。

$$\alpha_{final} = \sigma(\alpha_{raw}) = \frac{1}{1 + e^{-\alpha_{raw}}} \quad \left\{ \begin{array}{l} \alpha_{raw} = -9999.0 \\ \alpha_{final} \approx 0 \end{array} \right.$$

- 当目标高斯球的 $\alpha_i = 0$ 时，它自身的色彩贡献 $c_i \cdot 0 = 0$;同时它的透过率 $1 - \alpha_i = 1$ ，这意味着光线穿过它时不仅没有色彩残留，且能量百分之百穿透到了被遮挡的后方环境。

核心挑战二：源码框架语义解耦

➤ 原生管道的“只读前向渲染”限制



- 原生架构机制（2D 高亮逻辑）：LangSplat 原生管线采用“**渲染后查询（Post-rendering Query）**”范式。
- 其底层算子将 3D 特征投影为 2D 特征图，随后在 2D 像素层面进行相似度匹配与视觉高亮。

➤ 问题：

- 渲染生成的 **2D 像素** 已丢失空间深度与原生不透明度（Opacity）信息，在 2D 特征图上进行语义匹配与遮罩，仅能实现视觉覆盖，无法重现被遮挡的背景光场。
- **架构冲突**：原始 3DGS/LangSplat 采用端到端的静态前向渲染设计，缺乏在渲染前拦截并修改 **3D 粒子属性** 的中间态（Intermediate State）干预接口。
- **数据截断问题**：原生 Scene 类在初始化时默认读取标准 .ply 点云文件。但 .ply 格式仅支持基础几何与球谐颜色，无法存储 LangSplat 新引入的高维语言特征张量（lang_feat）。若不加干预，语言场将在加载阶段丢失。



构建支持状态干预的独立管线

为实现基于特征的场景编辑，在 Python 接口层对原生流水线进行了解耦重构：

装载层：基于状态字典的完整加载

- 隔离原生读取：强制 Scene 初始化空场景，阻断 .ply 的缺省读取逻辑。
- 读取 .pth 字典：选择直接使用 torch.load 读取训练生成的 .pth 检查点（State Dict）。因为只有该字典完整保留了训练期所有的自定义高维张量（包含 lang_feat 与原始 _opacity），以此构建出包含语义的完整工作区。

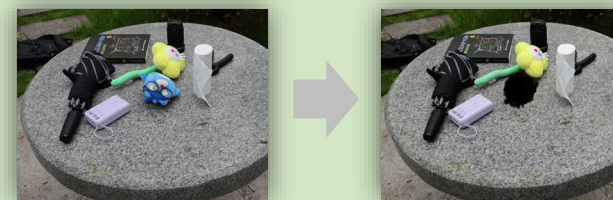
```
scene = Scene(dataset, gaussians, shuffle=False)
# 直接读取 .pth 状态字典，确保语言特征张量不丢失
model_params, _ = torch.load(checkpoint)
gaussians.restore(model_params, args, mode='test')
```

逻辑层：张量级状态干预

- 3D 原生对齐：彻底弃用原生管线的“2D 特征图渲染后查询”。在内存工作区中，直接将文本 CLIP 向量与三维空间内的高斯实体特征张量进行余弦相似度计算，提取真实的 3D 粒子拓扑索引（Mask）。
- 锁定目标 3D 高斯集群后，直接对其底层的物理不透明度（_opacity）张量执行极值覆写（-9999.0），从光学源头剥夺其遮挡能力。

执行层：上下文显式注入

- 避开原生训练管线，在外层重构并显式传入底层光栅化算子（render）强制依赖的全局配置对象（args），完成篡改后场景的单帧输出。



物理抹除后必然留下几何空洞（黑洞）

局限

展望

计划结合 2D 扩散模型（Diffusion Models）的先验知识，向 3D Inpainting（场景一致性补全）方向演进。